



Python Programming - 2301CS404

Lab - 8

Roll No : 310

Name : Ronit Bhadania

01) WAP to find their union, intersection, and difference of the given two sets.

Sample Input:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Sample Output:

```
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}
```

```
In [2]: a = int(input("Enter the size of set-1: "))
        b = int(input("Enter the size of set-2: "))

        s1 = set(map(int, input("Enter elements of set-1: ").split()))
        s2 = set(map(int, input("Enter elements of set-2: ").split()))

        print(f"Union: {s1.union(s2)}")
        print(f"Intersection: {s1.intersection(s2)}")
        print(f"Difference (s1 - s2): {s1.difference(s2)}")
```

```
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (s1 - s2): {1, 2}
```

02) WAP to Find elements common to all three given sets.

Sample Input:

A = {1, 2, 3, 4}
B = {2, 3, 5}
C = {2, 3, 6}

Sample Output:

{2, 3}

```
In [3]: a = int(input("Enter the size of set-1: "))
b = int(input("Enter the size of set-2: "))
c = int(input("Enter the size of set-3: "))

s1 = set(map(int, input("Enter elements of set-1: ").split()))
s2 = set(map(int, input("Enter elements of set-2: ").split()))
s3 = set(map(int, input("Enter elements of set-3: ").split()))

print(f"Intersection: {s1.intersection(s2,s3)}")
```

Intersection: {2, 3}

03) WAP to remove duplicate elements from a list while preserving unique values.**Sample Input:**

[10, 20, 10, 30, 20, 40]

Sample Output:

{10, 20, 30, 40}

```
In [7]: n = int(input("Enter the size of list: "))
l1 = list(map(int, input("Enter list elements: ").split()))
print(l1)
print(set(l1))
```

[10, 20, 10, 30, 20, 40]
{40, 10, 20, 30}

04) WAP to count how many elements are common between two sets.**Sample Input:**

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

2

```
In [11]: a = int(input("Enter the size of set-1: "))
b = int(input("Enter the size of set-2: "))
count=0
s1 = set(map(int, input("Enter elements of set-1: ").split()))
s2 = set(map(int, input("Enter elements of set-2: ").split()))

count = len(s1.intersection(s2))
print(count)
```

2

05) WAP to Merge two dictionaries. If a key exists in both dictionaries, add their values.

Sample Input:

```
Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
```

Sample Output:

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

```
In [1]: d1 = {}
d2 = {}

n1 = int(input("Enter the size of the dictionary-1: "))
for i in range(n1):
    k = int(input("Enter the key:"))
    v = int(input("Enter the value:"))
    d1[k] = v
n2 = int(input("Enter the size of the dictionary-2: "))
for i in range(n2):
    k = int(input("Enter the key:"))
    v = int(input("Enter the value:"))
    d2[k] = v

for k in d2:
    d1[k] = d1.get(k,0) + d2[k]

print(d1)
```

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

06) WAP to Sort a dictionary in ascending order of its values.

Hint: in sorted(), use udf for key parameter

Sample Input:

```
{'Math': 70, 'Physics': 85, 'Chemistry': 60}
```

Sample Output:

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

```
In [1]: def get_value(item):  
        return item[1]  
data = {'Math': 70, 'Physics': 85, 'Chemistry': 60}  
sorted_items = sorted(data.items(), key=get_value)  
sorted_dict = dict(sorted_items)  
print(sorted_dict)
```

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

07) WAP to find the keys having maximum and minimum values in a dictionary.**Sample Input:**

```
{'A': 50, 'B': 90, 'C': 30}
```

Sample Output:

```
Max Value Key: B  
Min Value Key: C
```

```
In [3]: d1 = {}  
n = int(input("Enter the size of the dictionary-1: "))  
for i in range(n):  
    k = input("Enter the key:")  
    v = int(input("Enter the value:"))  
    d1[k] = v  
  
max_key = max(d1, key=d1.get)  
min_key = min(d1, key=d1.get)  
  
print(f"Max Value Key: {max_key}")  
print(f"Min Value Key: {min_key}")
```

```
Max Value Key: B  
Min Value Key: C
```

08) WAP to create a dictionary with numbers from 1 to N as keys and their squares as values.**Sample Input:**

```
N = 5
```

Sample Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
In [4]: n = int(input("Enter the value of n:"))
d1 = {}
for i in range(1,n+1):
    d1[i] = i*i
print(d1)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

09) WAP to count the frequency of each character in a string using a dictionary.

Sample Input:

```
programming
```

Sample Output:

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

```
In [5]: s1 = input("Enter the string:")
d1 = {}
for i in s1:
    d1[i] = s1.count(i)
print(d1)
```

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

10) WAP to swap keys and values of a dictionary.

Sample Input:

```
{1: 'A', 2: 'B', 3: 'C'}
```

Sample Output:

```
{'A': 1, 'B': 2, 'C': 3}
```

```
In [8]: d1 = {}
n = int(input("Enter the size of the dictionary-1: "))
for i in range(n):
    k = int(input("Enter the key:"))
    v = input("Enter the value:")
    d1[k] = v
inv_di = {}
for k,v in d1.items():
    inv_di[v] = k
print(inv_di)
```

```
{'A': 1, 'B': 2, 'C': 3}
```

11) WAP to find the student with the highest total marks, given student names and their marks.

Sample Input:

```
{'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
```

Sample Output:

Topper: Amit

```
In [10]: n = int(input("Enter number of students: "))
students = {}

for i in range(n):
    name = input("Enter student name: ")
    marks = list(map(int, input("Enter the marks: ").split()))
    students[name] = marks

topper = ""
max_total = 0

for name, marks in students.items():
    total = sum(marks)
    if total > max_total:
        max_total = total
        topper = name

print(f"Topper: {topper}")
```

Topper: Amit

12) WAP to extract keys whose values are even numbers.**Sample Input:**

```
{1: 11, 2: 20, 3: 15, 4: 40}
```

Sample Output:

```
[2, 4]
```

```
In [12]: d1 = {}
n = int(input("Enter the size of the dictionary-1: "))
for i in range(n):
    k = int(input("Enter the key:"))
    v = int(input("Enter the value:"))
    d1[k] = v

even_values = []

for k,v in d1.items():
    if v%2 == 0:
        even_values.append(k)
print(even_values)
```

```
[2, 4]
```

13) WAP to Find common keys between two dictionaries.

Sample Input:

```
Dict1 = {1: 'A', 2: 'B', 3: 'C'}  
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}
```

Sample Output:

```
[2, 3]
```

```
In [13]: d1 = {}  
d2 = {}  
n1 = int(input("Enter the size of the dictionary-1: "))  
for i in range(n1):  
    k = int(input("Enter the key:"))  
    v = input("Enter the value:")  
    d1[k] = v  
n2 = int(input("Enter the size of the dictionary-1: "))  
for i in range(n2):  
    k = int(input("Enter the key:"))  
    v = input("Enter the value:")  
    d2[k] = v  
common_keys = []  
  
for k in d1:  
    if k in d2:  
        common_keys.append(k)  
print(common_keys)
```

```
[2, 3]
```

14) WAP to handle missing keys in dictionaries.

Sample Input - 1:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}  
key = 'c'
```

Sample Output - 1:

```
8
```

Sample Input - 2:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}  
key = 'd'
```

Sample Output - 2:

```
Key Not Found
```

```
In [15]: d1 = {}  
n = int(input("Enter the size of the dictionary:"))  
for i in range(n) :
```

```
k = input("Enter the key:")
v = int(input("Enter the value:"))
d1[k] = v
search_key = input("Enter key to search: ")
if search_key in d1:
    print(d1[search_key])
else :
    print("Key Not Found")
```

8

15) WAP to create a dictionary using dictionary comprehension from two lists containing student names and marks. Include only those students in the dictionary whose marks are greater than or equal to 75.

Sample Input:

```
students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
```

Sample Output:

```
{'Amit': 78, 'Neha': 85}
```

```
In [16]: n = int(input("Enter number of students: "))
students = []
marks = []
for i in range(n):
    students.append(input("Enter students name: "))
    marks.append(int(input("Enter marks: ")))
result = {students[i]: marks[i] for i in range(n) if marks[i] >= 75}
print(result)
```

```
{'Amit': 78, 'Neha': 85}
```